Activity 1:- Certificate Analysis and examining the SSL/TLS handshaking procedure by using Wireshar

Introduction

This report analyzes HTTPS traffic captured using Wireshark to study communication with `https://www.wikipedia.org`. The steps involved include:

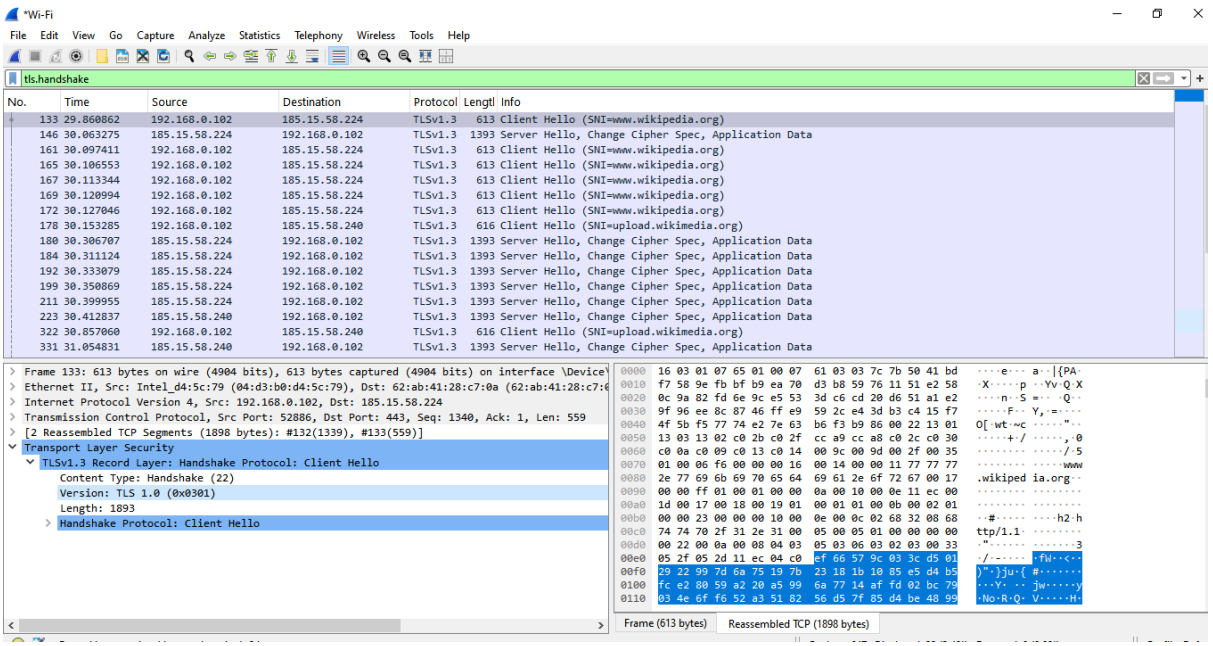
1. Capturing network traffic while visiting the website using Wireshark.
2. Applying a display filter (`tcp.port == 443`) to isolate HTTPS traffic.
3. Interacting with the target website in a web browser.
4. Stopping and saving the captured traffic for analysis.

The goal is to identify and analyze packets specific to `https://www.wikipedia.org` by filtering based on IP addresses and examining TLS and HTTPS protocol details.

2. Analysis the Trace

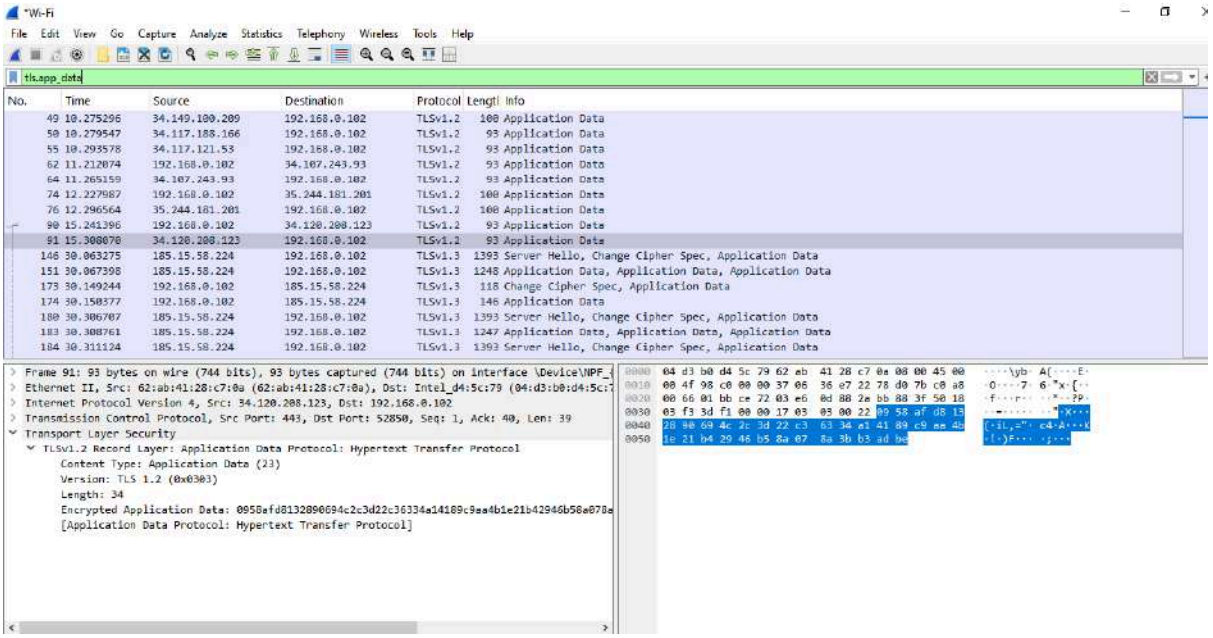
Answer the following Questions based on the above trace and also provide an image for each question to support your answers:-

Q1. What number identifies the SSL Handshake content type?



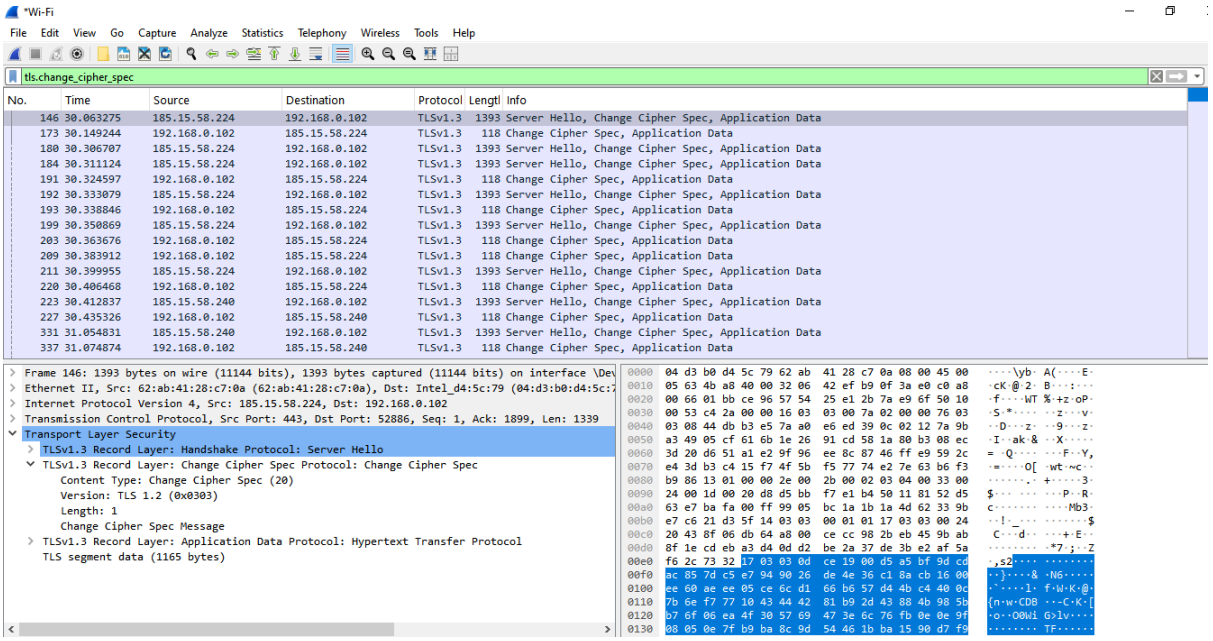
`Content Type: Handshake (22)` in a `ClientHello` packet.

Q2. What number identifies the SSL Application Data content type?



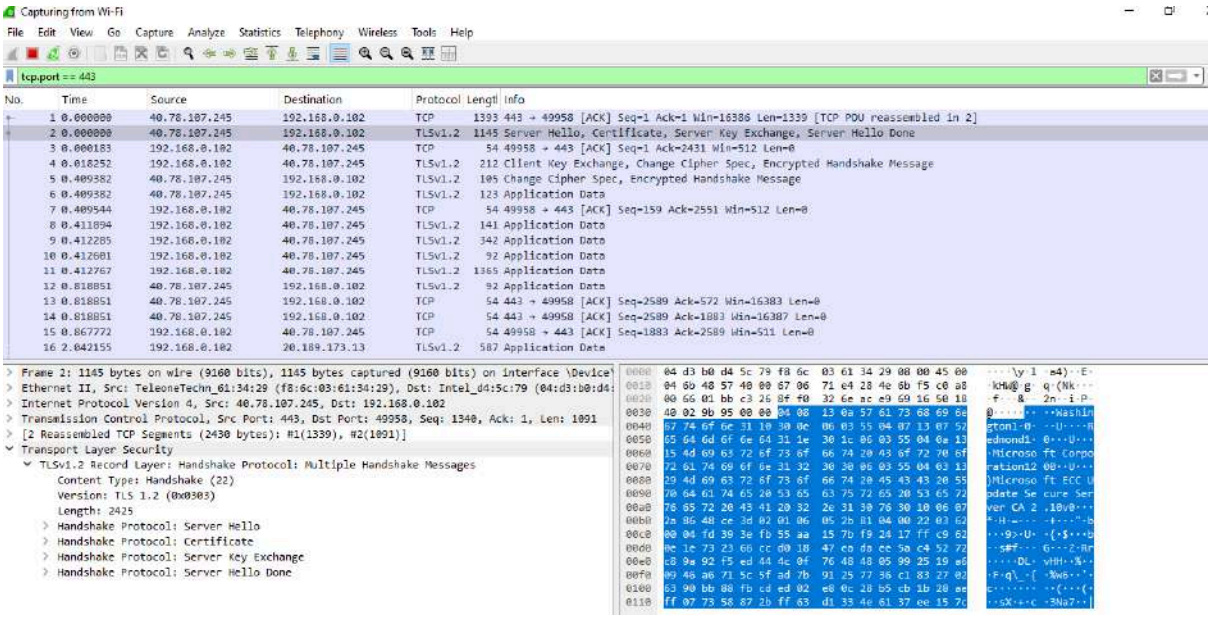
content type for SSL Application Data is 23

Q3. What number identifies the SSL Change Cipher Spec content type?



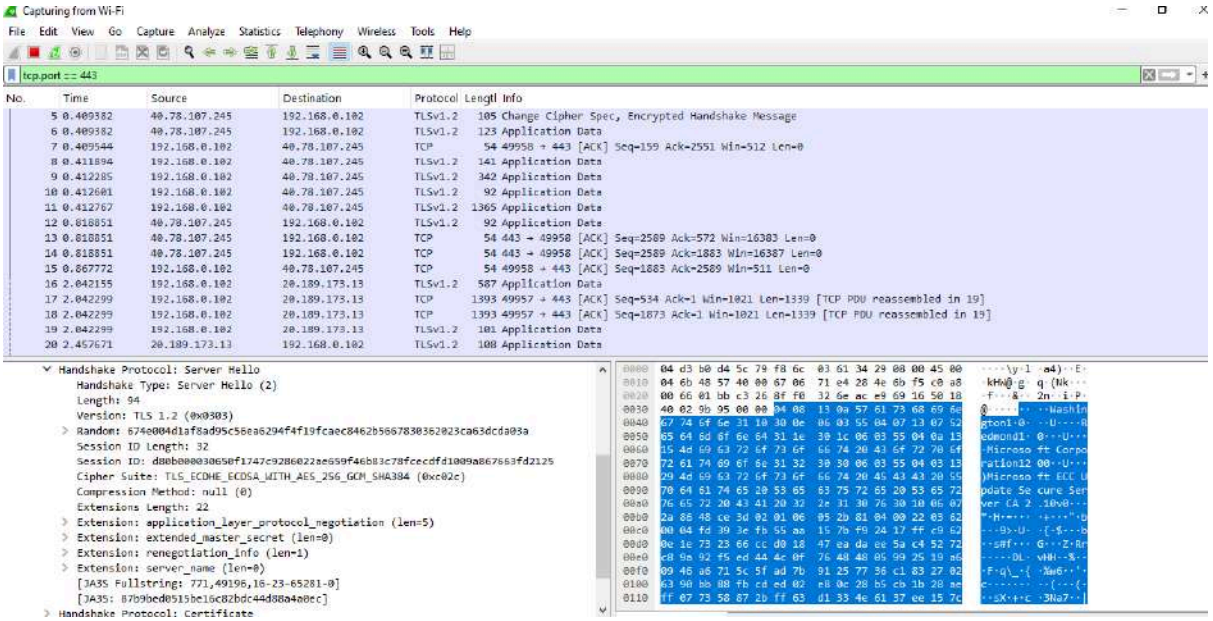
content type for SSL Change Cipher Spec is 20.

Q4. In your Wire shark trace, what all sets of messages is bundled together into single frames?

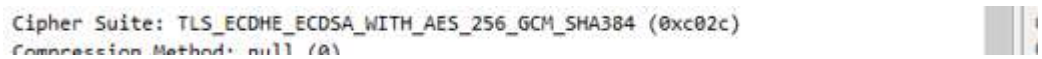


I get Bundled Messages in Single Frames bundle. For instance:- ServerHello, Certificate, and ServerHelloDone in a single frame.

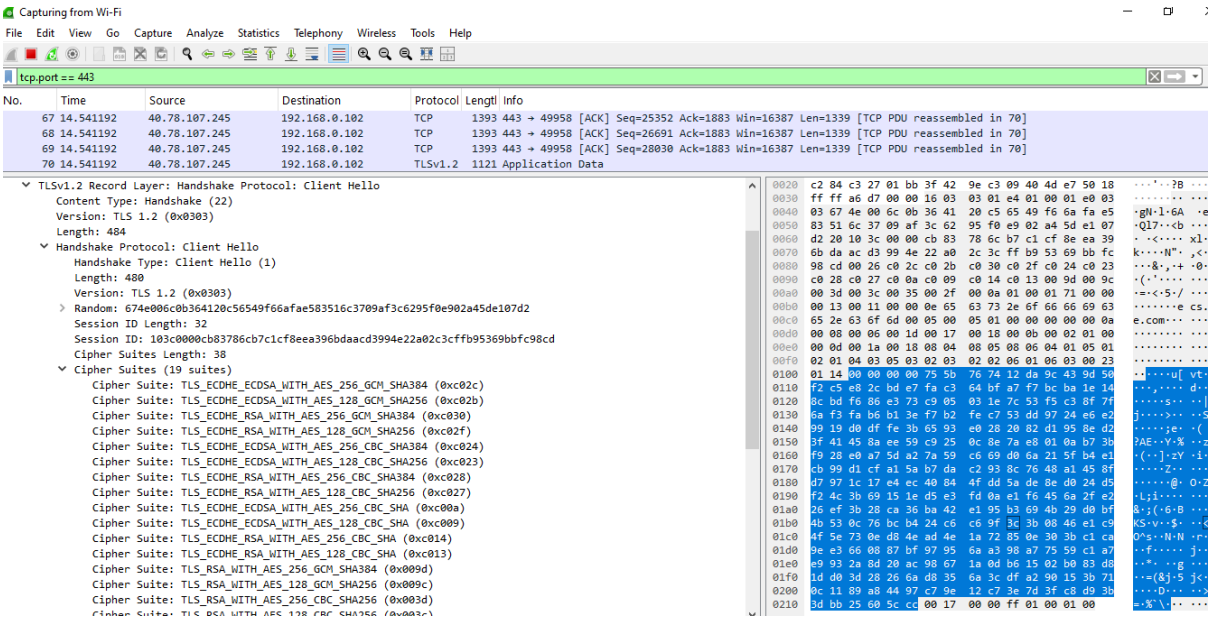
Q5. What is the cipher suite selected for this session?



Based on the above I selected the Cipher Suite that located on Cipher Suite field.

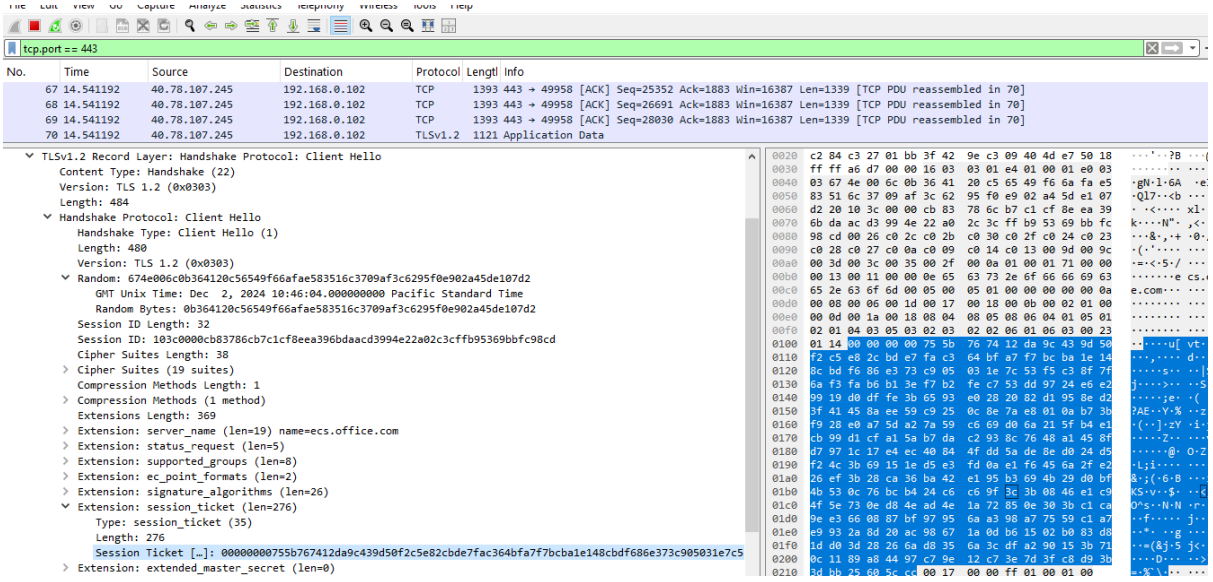


Q6. What are the first 5 cipher suites, suggested by the client machine?

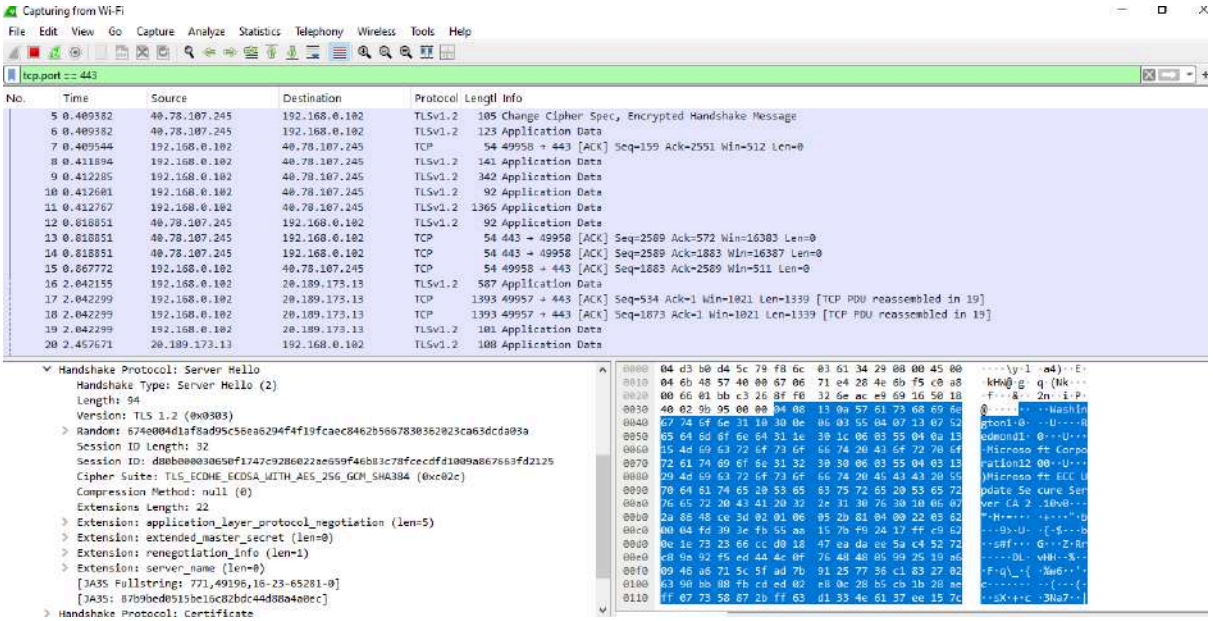


Here is the first 5 cipher suites that suggest by client machine

Q7. What is ClientRandomValue included in the Client Hello message?



Q8. Is it possible for you to learn the pre-master secret from this trace? If yes, provide the pre-master secret. If not, explain your answer.



Based on the above image it includes ECDHE (Ephemeral Diffie-Hellman), **pre-master secret is not accessible** because of Perfect Forward Secrecy (PFS).

Q9. Establish another (fresh) session with your chosen website, (close the browser and start a new session from the same browser), and identify whether the answers to each of the above questions (Q1-Q8) change or not?

When to test the other website in the same web browser and to filter it on the wireshark.

Based on the below image is Capture a new trace and compare with the above website

- **ClientRandom** and other session-specific values (like session keys) will change.
- **Content types** and supported cipher suites are likely unchanged.

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.port == 443

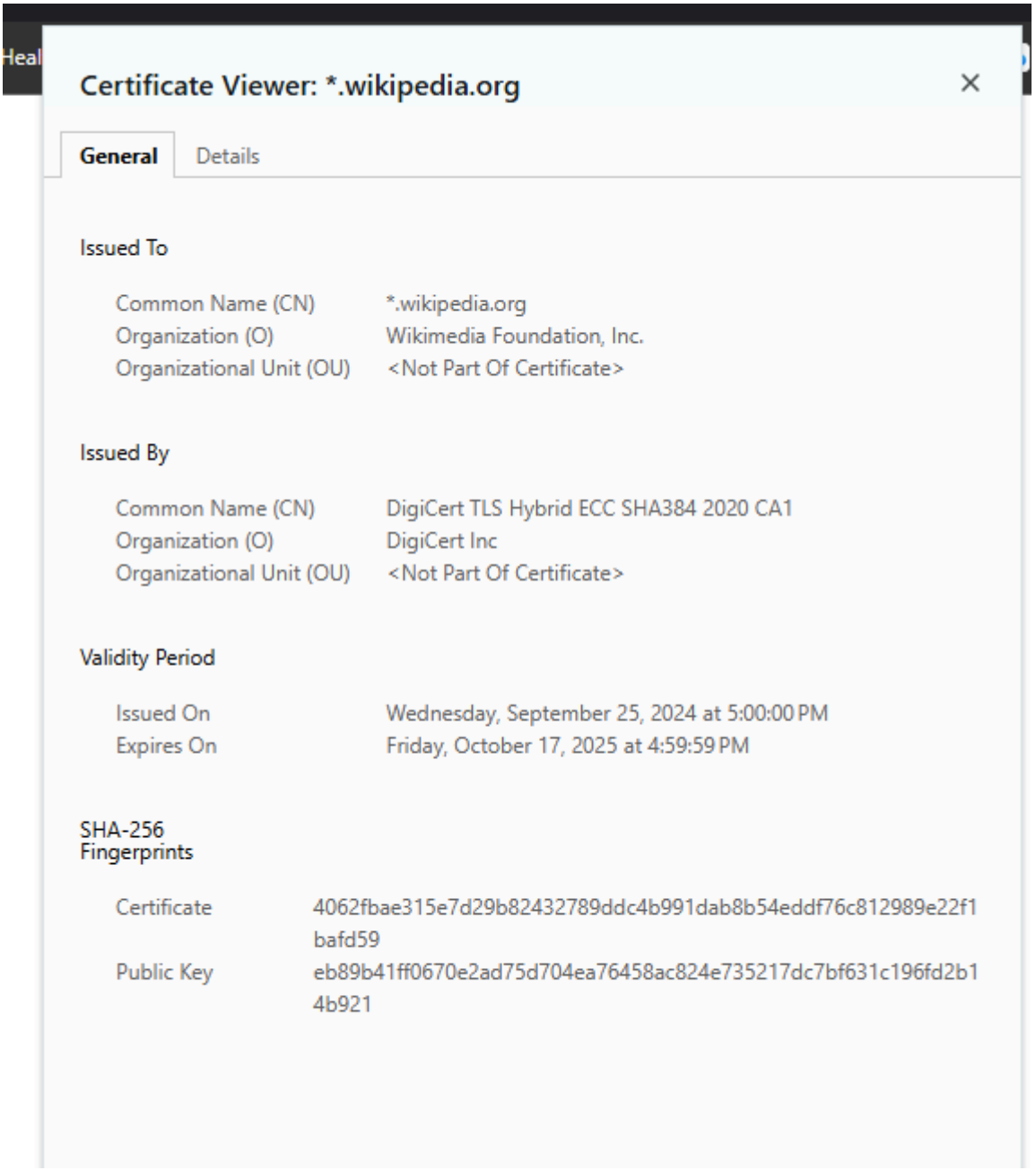
No.	Time	Source	Destination	Protocol	Length	Info
1680	8.167044	23.207.19.74	192.168.0.102	TCP	66	[TCP Dup ACK 1637#1] 443 → 50078 [ACK] Seq=22062 Ack=2439 Win=64128 Len=0 SLE=2408 SRE=2439
1682	8.245676	67.220.228.200	192.168.0.102	TCP	212	[TCP Spurious Retransmission] 443 → 50085 [PSH, ACK] Seq=6608 Ack=1801 Win=61056 Len=158
1683	8.245730	192.168.0.102	67.220.228.200	TCP	66	[TCP Dup ACK 1664#1] 50085 → 443 [ACK] Seq=1801 Ack=6766 Win=131840 Len=0 SLE=6608 SRE=6766
1687	8.327685	18.245.143.118	192.168.0.102	TCP	54	443 → 50079 [ACK] Seq=18207 Ack=2519 Win=72192 Len=0
1688	8.336740	52.84.90.40	192.168.0.102	TCP	54	443 → 50077 [ACK] Seq=7127 Ack=2447 Win=72192 Len=0
1696	8.535459	192.168.0.102	184.28.85.210	TCP	66	50086 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1701	8.718720	184.28.85.210	192.168.0.102	TCP	66	443 → 50086 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1339 SACK_PERM WS=128
1702	8.718839	192.168.0.102	184.28.85.210	TCP	54	50086 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=0
1703	8.720519	192.168.0.102	184.28.85.210	TCP	1393	50086 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=1339 [TCP PDU reassembled in 1704]
1704	8.720519	192.168.0.102	184.28.85.210	TLSv1.3	511	Client Hello (SNI=eus.rubiconproject.com)
1709	8.936835	184.28.85.210	192.168.0.102	TLSv1.3	1393	Server Hello, Change Cipher Spec, Application Data
1710	8.941694	184.28.85.210	192.168.0.102	TCP	1393	443 → 50086 [PSH, ACK] Seq=1340 Ack=1797 Win=64128 Len=1339 [TCP PDU reassembled in 1711]
1711	8.941694	184.28.85.210	192.168.0.102	TLSv1.3	1024	Application Data, Application Data, Application Data
1712	8.941803	192.168.0.102	184.28.85.210	TCP	54	50086 → 443 [ACK] Seq=1797 Ack=3649 Win=131072 Len=0
1713	8.946118	184.28.85.210	192.168.0.102	TCP	54	443 → 50086 [ACK] Seq=1 Ack=1340 Win=64128 Len=0
1714	8.946118	184.28.85.210	192.168.0.102	TCP	54	443 → 50086 [ACK] Seq=1 Ack=1797 Win=64128 Len=0
1715	8.946183	192.168.0.102	184.28.85.210	TCP	54	[TCP Dup ACK 1712#1] 50086 → 443 [ACK] Seq=1797 Ack=3649 Win=131072 Len=0
1716	8.946334	192.168.0.102	184.28.85.210	TCP	54	[TCP Dup ACK 1712#2] 50086 → 443 [ACK] Seq=1797 Ack=3649 Win=131072 Len=0

```
Transport Layer Security
  TLSv1.3 Record Layer: Handshake Protocol: Client Hello
    Content Type: Handshake (22)
    Version: TLS 1.0 (0x0301)
    Length: 1791
    Handshake Protocol: Client Hello
      Handshake Type: Client Hello (1)
      Length: 1787
      Version: TLS 1.2 (0x0303)
      Random: 017cc68caac1f5d4fa96d7d089f78bd33b06e9e6ce15fc75f204277fe2702950
      Session ID Length: 32
      Session ID: f49ed347a74c7aa85209759ae4e67bc5c44fc6f24b015ba23f7f4c90bef2f2a5
      Cipher Suites Length: 32
      Cipher Suites (16 suites)
        Cipher Suite: Reserved (GREASE) (0x7a7a)
```

```
0000  f8 6c 03 61 34 29 04 d3  b0 d4 5c 79 08 00 45 00   ·l·a4)·· ··\y··E·
0010  01 f1 7a c4 40 00 80 06  af 45 c0 a8 00 66 b8 1c   ··z·@··· ·E···f··
0020  55 d2 c3 a6 01 bb 03 03  0c df ed f0 f3 4f 50 18   U····· ·····OP·
0030  02 00 86 54 00 00 94 ee  a6 66 fc 27 8f 7f 45 7e   ···T···· ·f·'··E~
0040  be c5 3c 98 b3 b3 99 32  4c 18 9c 0a fc 48 6f b5   ··<····2 L····Ho·
0050  a4 4e 66 73 07 8f c0 40  16 81 ae 4b 74 69 96 69   ·Nfs···@ ···Kti·i
0060  6d 21 7b 4d e6 26 0b d4  cb 23 b8 32 76 43 85 5c   m!{M·&·· ·#·2vC·\
0070  7d 89 56 c5 a5 73 00 c5  27 fb f5 9e ec 71 a6 7f   }·V··s·· '····q··
0080  e0 1d c8 58 a3 2a 08 0f  d9 64 23 af 58 bc eb e1   ···X·*·· ·d#·X···
0090  10 92 30 60 36 a2 5e b0  ea 11 9e 18 32 63 a8 2c   ··0`6·^· ····2c·,
00a0  44 5a 62 e6 f2 ae 8b c0  1e 28 fc 48 16 21 69 c4   DZb····· ·(·H·!i·
00b0  40 96 8d b5 80 4f 36 b0  27 6b 28 2e 8b 5d de 06   @····O6· 'k(··]··
00c0  c1 32 f8 c6 ac 10 c7 cb  33 bf 2b 38 43 65 23 55   ·2····· 3·+8Ce#U
00d0  46 b9 c0 b9 c2 72 0a 57  37 fe 08 94 9b 75 c0 05   F····r·W 7····u··
00e0  7a 3e 16 82 a1 08 54 71  53 f6 a1 af 3b cc 28 47   z>····Tq S···;·(G
```

3. Certificate Analysis

I analysis the certificate of the website https://www.wikipedia.org/ and To view a certificate on Chrome



Certificate Viewer: *.wikipedia.org

General | Details

Issued To

Common Name (CN)	*.wikipedia.org
Organization (O)	Wikimedia Foundation, Inc.
Organizational Unit (OU)	<Not Part Of Certificate>

Issued By

Common Name (CN)	DigiCert TLS Hybrid ECC SHA384 2020 CA1
Organization (O)	DigiCert Inc
Organizational Unit (OU)	<Not Part Of Certificate>

Validity Period

Issued On	Wednesday, September 25, 2024 at 5:00:00 PM
Expires On	Friday, October 17, 2025 at 4:59:59 PM

SHA-256 Fingerprints

Certificate	4062fbae315e7d29b82432789ddc4b991dab8b54eddf76c812989e22f1bafd59
Public Key	eb89b41ff0670e2ad75d704ea76458ac824e735217dc7bf631c196fd2b14b921

4. Analyze the certificate of any website by using all methods mentioned above and Answer the following Question:

a. Who has signed the certificate, i.e., who is the CA? Is the CA trusted by your browser?

- **Certificate Authority (CA):** The certificate is signed by **DigiCert TLS Hybrid ECC SHA384 2020 CA1**.
- **Trusted by Browser:** Yes, DigiCert is a well-known trusted CA. Since the browser displays the certificate without any warnings, it indicates the CA is trusted.

b. What is the certification authority, hierarchy for the certificate?

The hierarchy can be inferred from the details:

1. **Root CA:** DigiCert Inc.
2. **Intermediate CA:** DigiCert TLS Hybrid ECC SHA384 2020 CA1.
3. **Issued to:** *.wikipedia.org (wildcard certificate for all subdomains of wikipedia.org).

c. Has the certificate expired?

Validity Period:

- **Issued On:** Wednesday, September 25, 2024, at 5:00:00 PM.
- **Expires On:** Friday, October 17, 2025, at 4:59:59 PM.

Current Date: Since today's date is **December 3, 2024**, the certificate has **not expired**.

d. What digital signature scheme is used to sign the certificate?

- The certificate uses the **SHA384** hash algorithm as part of the signature scheme. From the name of the issuer, the signature scheme is likely **ECDSA with SHA384** (Elliptic Curve Digital Signature Algorithm).

e. What cryptosystem is used to generate the public (private) keys for the subject, regions?

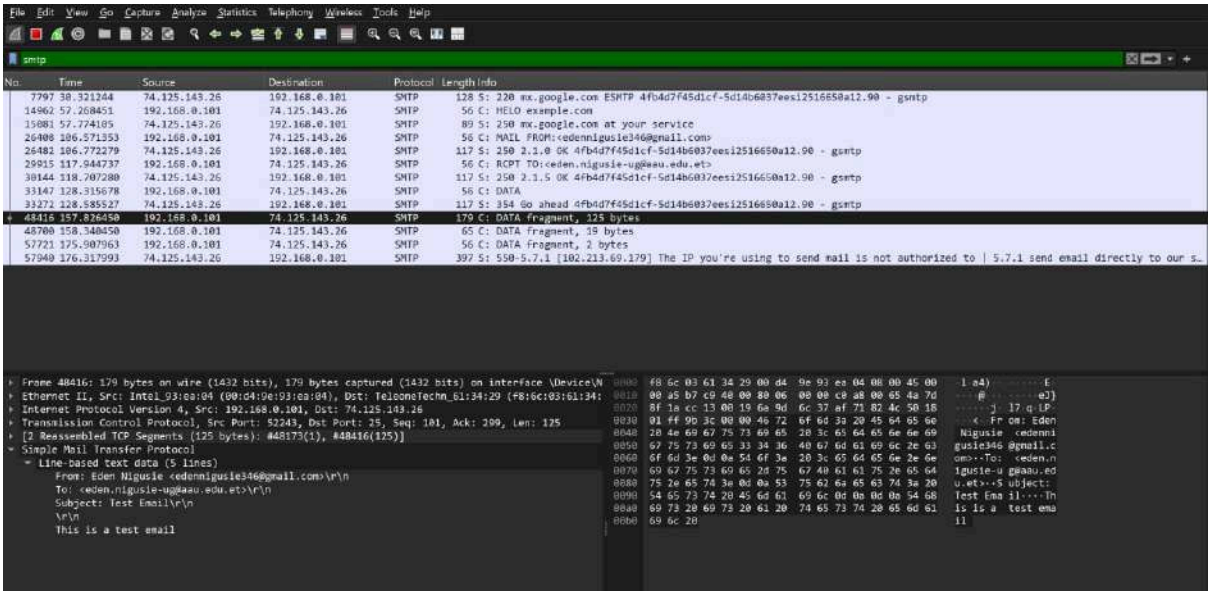
- The cryptosystem used is **Elliptic Curve Cryptography (ECC)**, as indicated by the issuer name (ECC SHA384) and the type of certificate.

f. What is the public key that"s been certified (just copy-paste it)? Public Key:

- eb89b41ff0670e2ad75d704ea76458ac824e735217dc7bf631c196fd2b14b921

Activity 2:- Examine the SMTP traffic using Wireshark

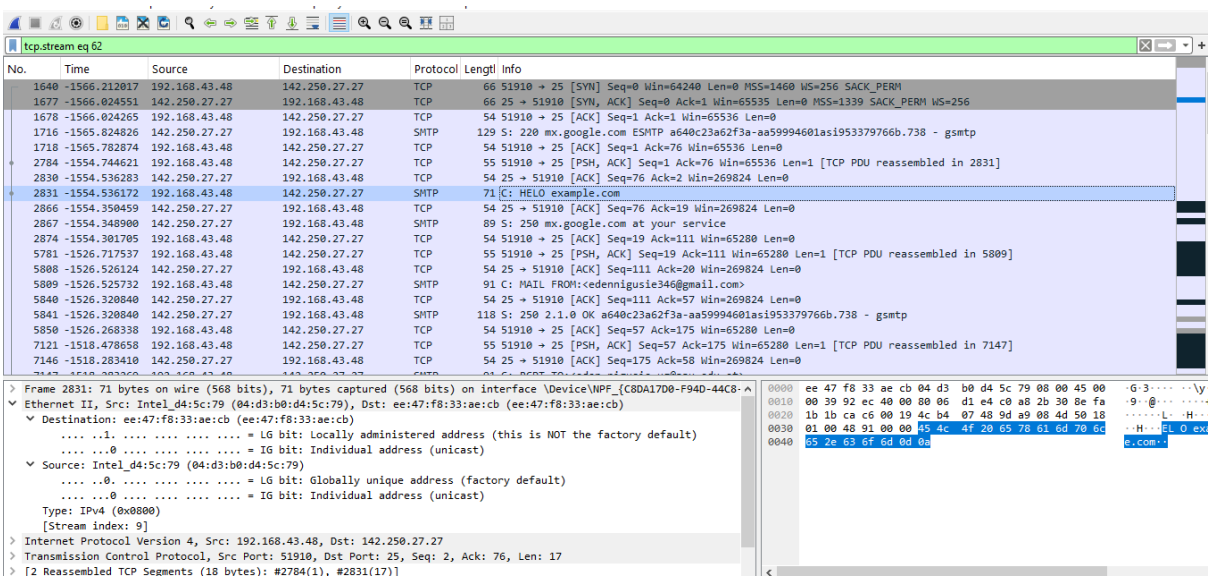
1. Capture the SMTP traffic. Provide an image to support your answers.



Here is the capture the SMTP traffic

2. Answer the following Question and also provide an image to support your answer:-

Q1. What is the Ethernet address of the SMTP client, and what is the Ethernet address of the SMTP server?

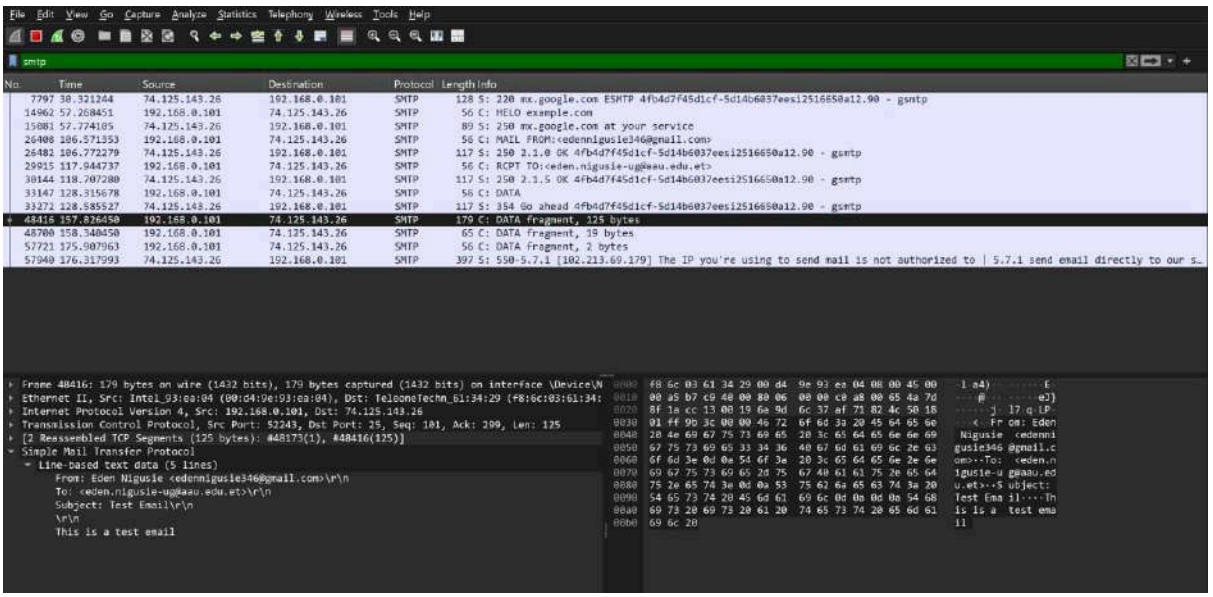


The Ethernet (MAC) addresses can be observed in the **Ethernet II** section of the packet details:

1. **SMTP Client Ethernet Address** (Source): **04:d3:b0:d4:5c:79**
2. **SMTP Server Ethernet Address** (Destination): **ee:47:f8:33:ae:cb**

These addresses represent the physical (MAC) addresses of the SMTP client and server.

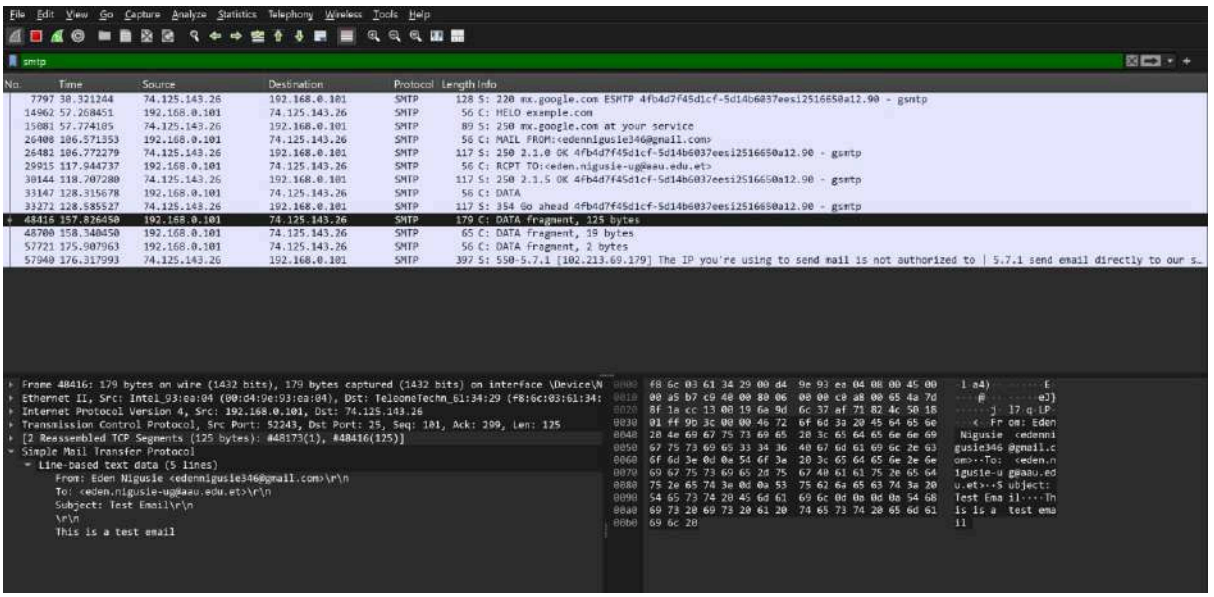
Q2. What is the IP address of the SMTP client, and what is the IP address of the SMTP server?



1. **SMTP Client IP Address** (Source): **192.168.0.101**
2. **SMTP Server IP Address** (Destination): **74.125.143.26**

These IP addresses represent the client (local device sending the email) and the server (SMTP server handling the email transmission).

Q3. Which frame was used to generate the To: header line in the message?

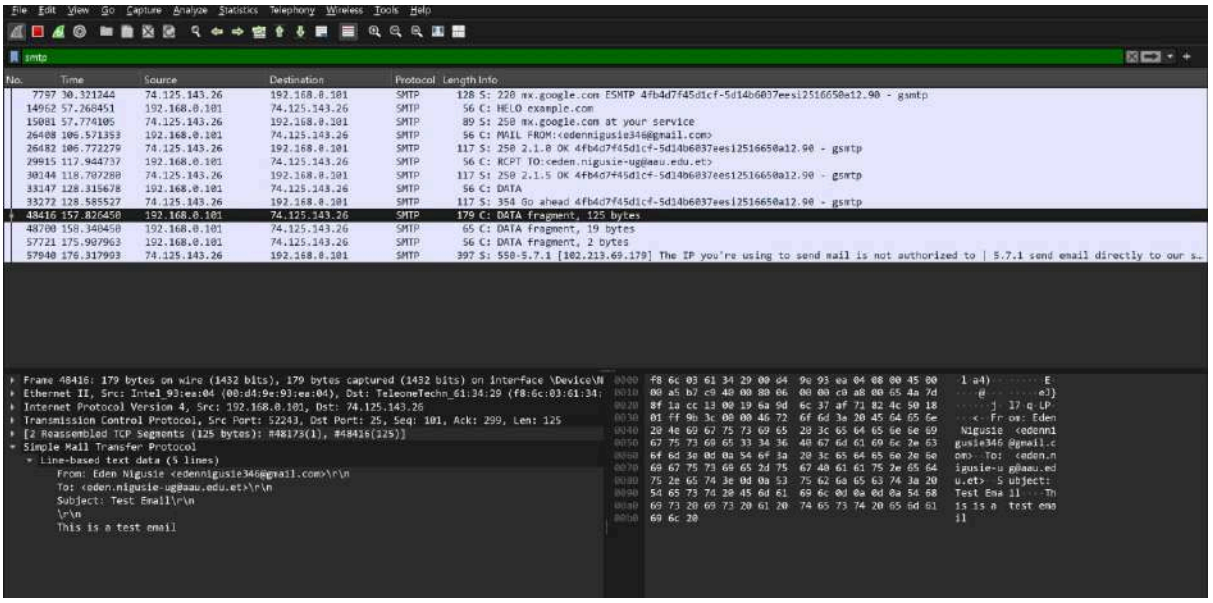


Frame 48414 was used to generate the **To: header line** in the SMTP message. This frame contains the following header line:

To: <eden.nigusie-u@aau.edu.et>

It indicates the recipient's email address in the SMTP conversation.

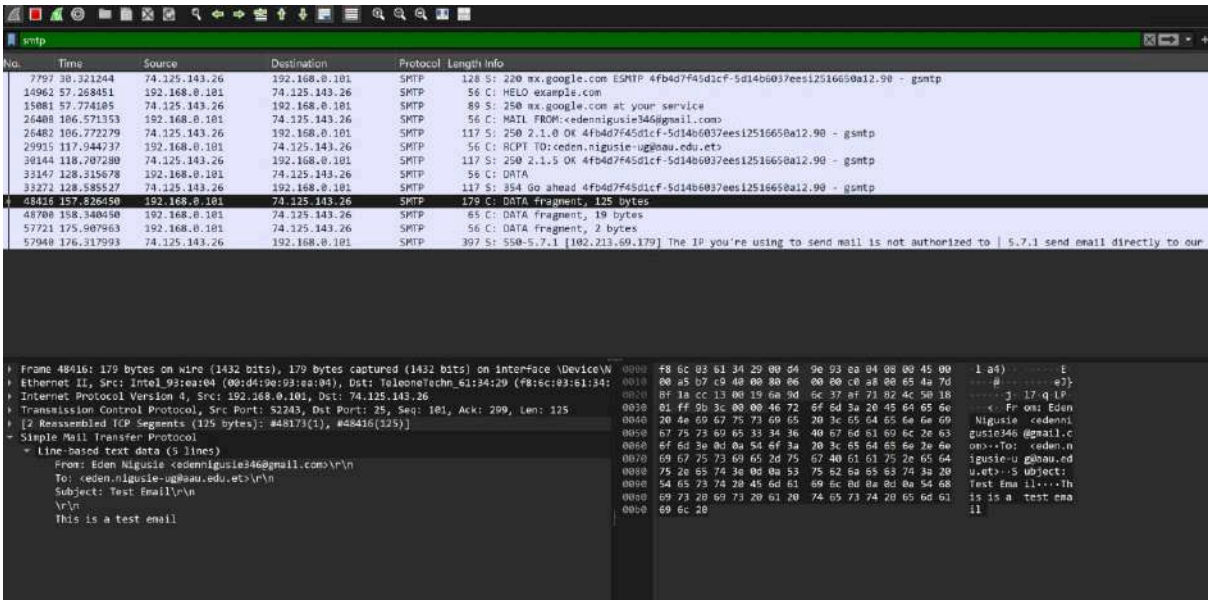
Q4. Which frame acknowledged the Subject: header line in the message?



Frame 48416 acknowledged the **Subject: header line** in the SMTP message. It includes the acknowledgment for the message content, confirming the reception of the previously sent Subject: header:

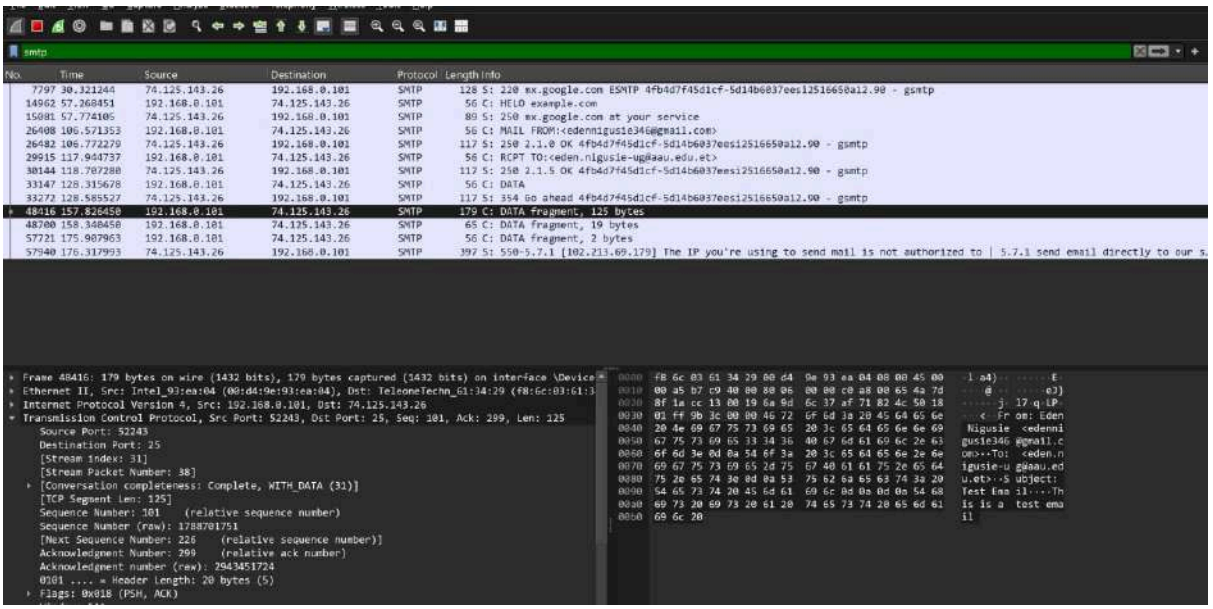
Subject: Test Email

Q5. Which frames generated the body of the message?



Frame 48416: This frame contains the initial part of the message body, as shown in the detailed payload (This is a test email).

Q6. What TCP port was used on SMTP client, and what TCP port was used on the SMTP server?



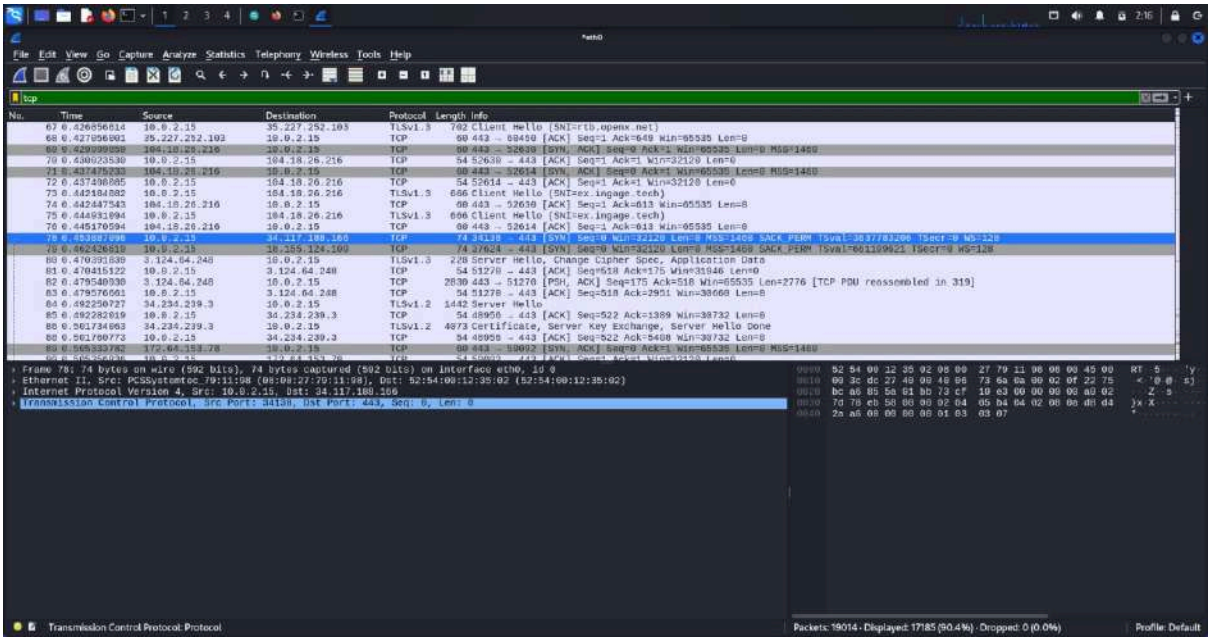
- The **SMTP client** used **TCP port 52243** (source port).
- The **SMTP server** used **TCP port 25** (destination port), which is the standard port for SMTP communication.

Q7. Why does Wireshark label some of the frames as TCP and some as SMTP?

It shows frames labeled as **TCP** for segments such as acknowledgment or fragments of data and as **SMTP** for identified email-related commands or messages (e.g., HELO, MAIL FROM, DATA). This demonstrates how Wireshark differentiates based on the data content and protocol layer.

Activity 3:- Examine the TCP packet using Wireshark

1.Examine the TCP packet



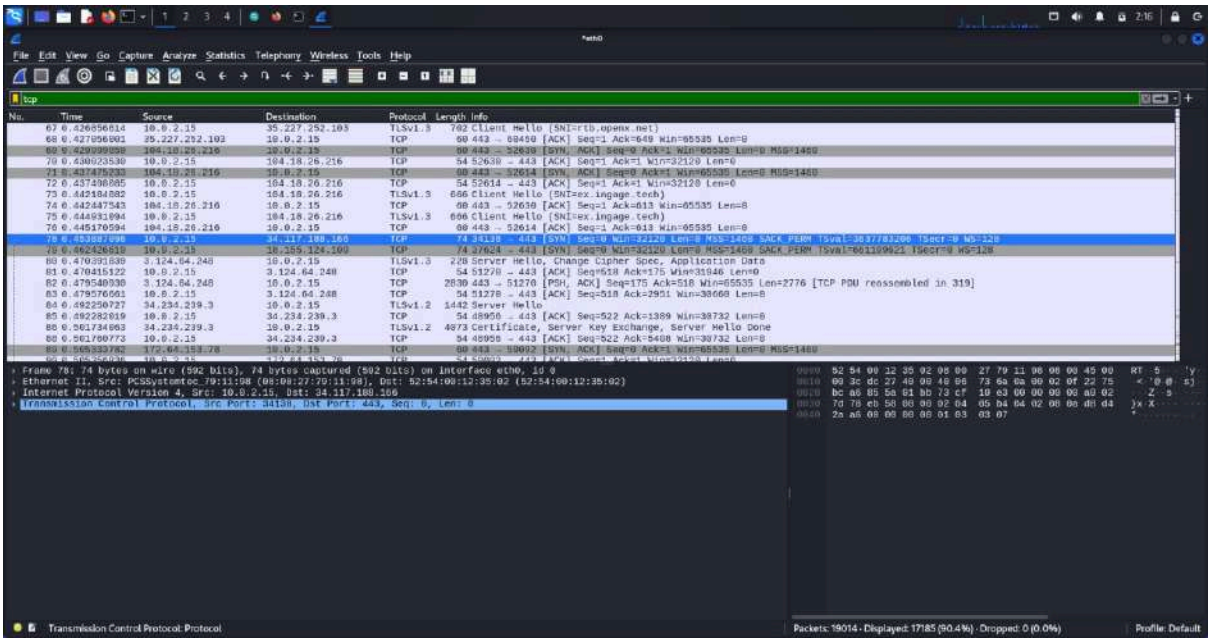
2. Answer the following questions and provide an image to support your answers:-

Q1. What is the IP address and TCP port number used by the client computer (source)?

IP address of the client computer (source): 10.0.2.15

TCP port number used by the client (source): 34138

Q2. What is the IP address of remote server? On what port number it is sending and receiving TCP segments for this connection?

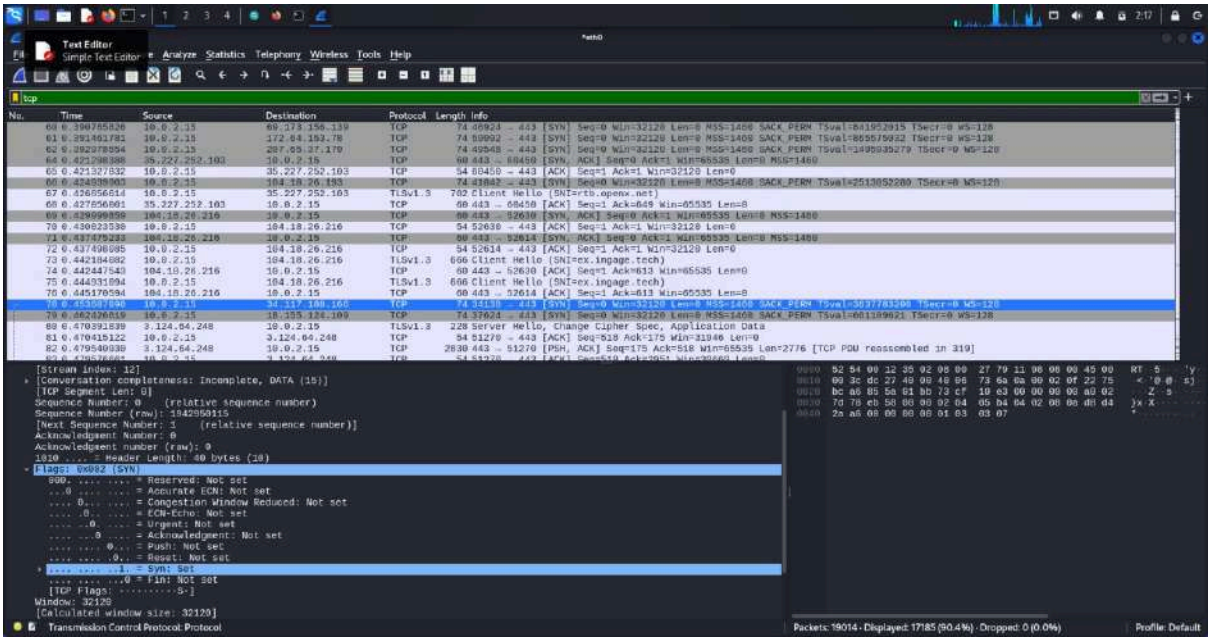


IP address of the remote server (destination): 34.117.188.166

Port number used by the remote server: 443 (commonly used for HTTPS)

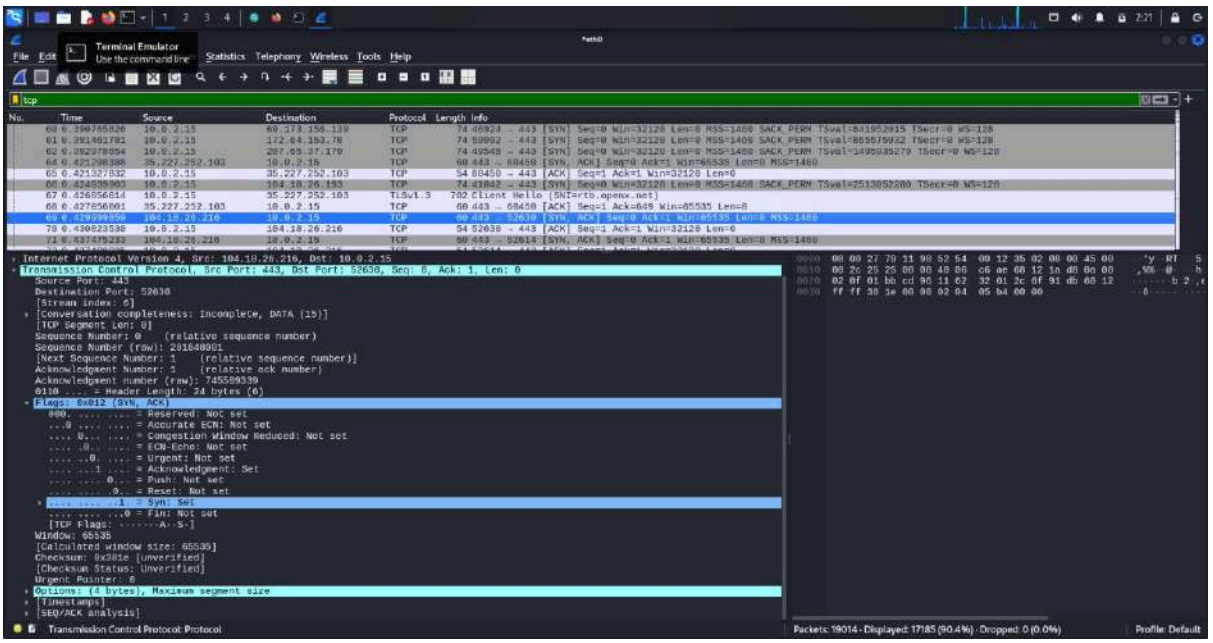
It is visible in the highlighted row, where the destination is 34.117.188.166 on port 443.

Q3. What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and the server? What is it in the segment that identifies the segment as a SYN segment?



- **Sequence Number**: The sequence number of the TCP SYN segment is 1949520115. This is displayed under the "Sequence Number (raw)" field in the packet details.
- **Identifying SYN Segment**: The segment is identified as a SYN segment by examining the TCP header flags. In this case, the "Flags" field shows 0x002, and under the decoded breakdown of flags, the "SYN" flag is set to 1 (indicated as "Set").

Q4. What is the sequence number of the SYNACK segment sent by the server to the client computer in reply to the SYN? What is the value of the Acknowledgement field in the SYNACK segment? How did server determine that value? What is it in the segment that identifies the segment as a SYNACK segment



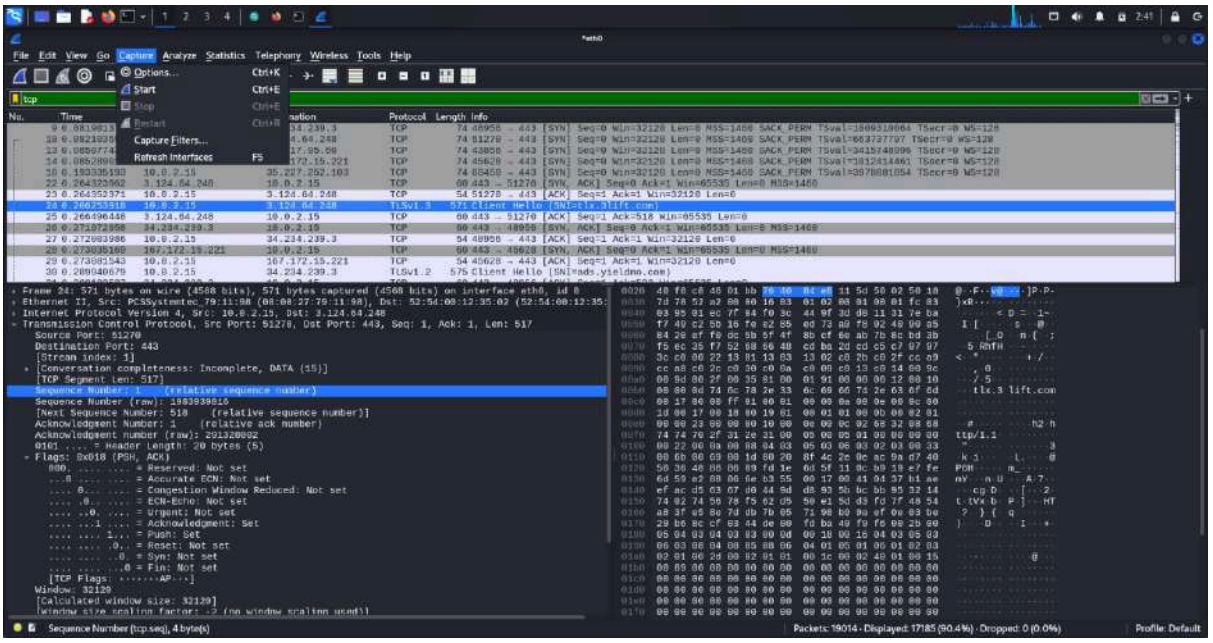
- The sequence number of the SYN-ACK segment is 2914084800. This is shown in the "Sequence Number (raw)" field in the packet details.

- The value of the acknowledgment field is 1949520116. This is shown in the "Acknowledgment Number (raw)" field.

- The acknowledgment field value is determined by adding 1 to the sequence number of the SYN segment sent by the client. In this case, the SYN segment's sequence number was 1949520115, so the acknowledgment field is 1949520115 + 1 = 1949520116.

- The SYN-ACK segment is identified by examining the TCP header flags. The "Flags" field shows 0x012, and in the breakdown, both the "SYN" and "ACK" flags are set to 1 (indicated as "Set"). This combination of flags designates the packet as a SYN-ACK segment.

Q5. What is the sequence number of the TCP segment containing the HTTP POST command?



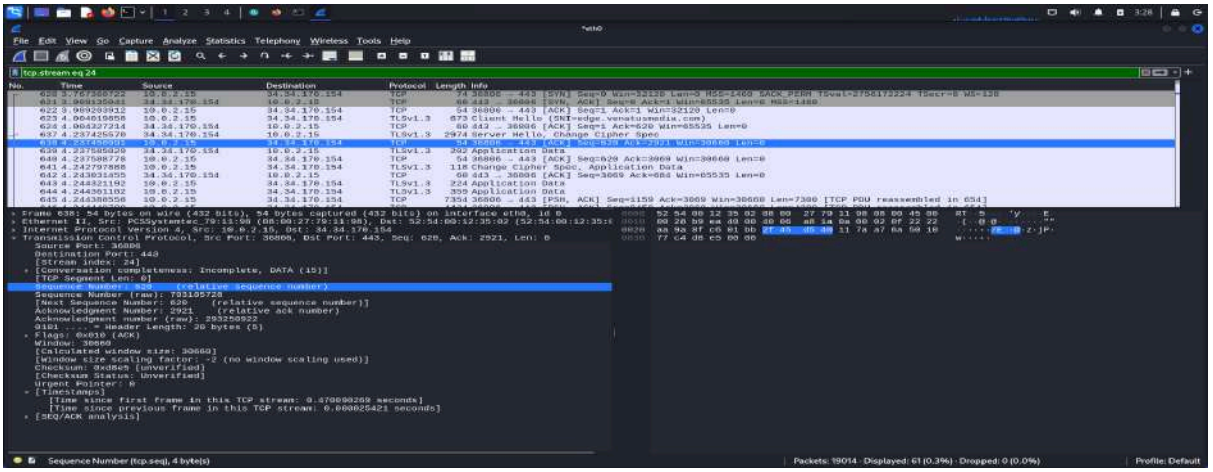
- **Sequence Number**: The sequence number of the TCP segment containing the HTTP POST command is 1938393916. This is displayed in the "Sequence Number (raw)" field in the packet details.
- **Identification**: The TCP segment containing the HTTP POST command is identifiable in the packet capture by examining its payload and sequence of events in the TCP stream. The payload typically starts with the HTTP POST header, visible in the raw data pane on the right.

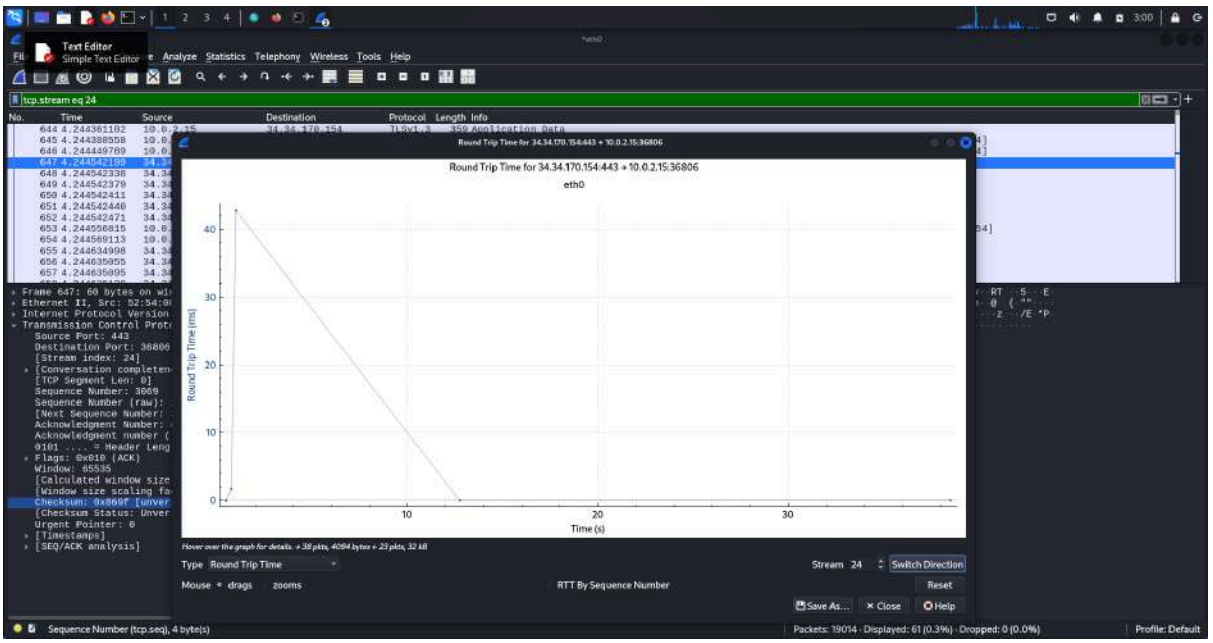
Q6. Consider the TCP segment containing the HTTP POST as the first segment in the TCP connection. What are the sequence numbers of the first six segments in the TCP connection. At what time was each segment sent? When was the ACK for each segment received? Given the difference between when each TCP segment was sent, and when its acknowledgement was received, what is the RTT value for each of the six segments?

> Note: Wireshark has a nice feature that allows you to plot the RTT for each of the TCP segments sent. Select a TCP segment in the “listing of captured packets” window that is being sent from the client to the server. Then select: Statistics->TCP Stream Graph>Round Trip Time Graph.

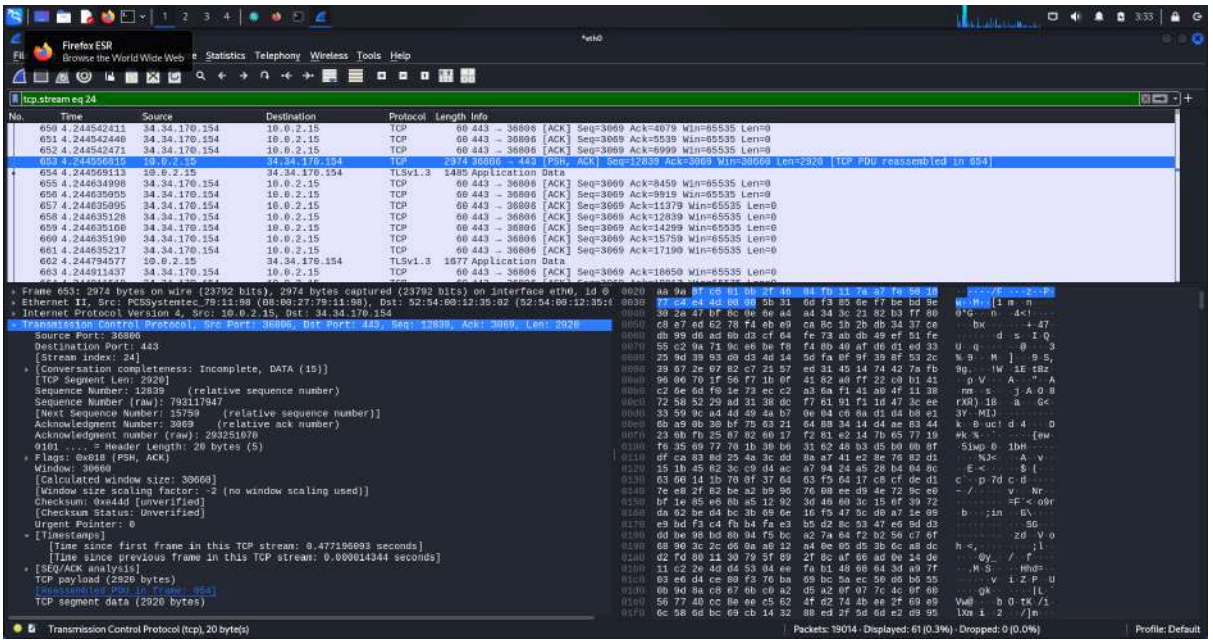
Segment 1 sequence number: 1
Segment 2 sequence number: 620
Segment 3 sequence number: 2921
Segment 4 sequence number: 3069
Segment 5 sequence number: 8459
Segment 6 sequence number: 12839

	Sent time	ACK received time	RTT in second
Segment 1	0.0148	0.2366	0.2218
Segment 2	0.000025	0.4700	0.4099
Segment 3	0.00013	0.4701	0.4699
Segment 4	0.00023	0.47567	0.47544
Segment 5	0.0000612	0.47708	0.477018
Segment 6	0.00001434	0.477196	0.47715166





Q7. What is the length of each of the first six TCP segments?
Solution: Length of the first TCP segment (containing the HTTP POST): 2920 bytes



Q8. What is the throughput (bytes transferred per unit time) for the TCP connection? Explain how you calculated this value.

The computation of TCP throughput depends significantly on the choice of the averaging time period. In this case, the entire duration of the connection is used as the averaging time period. To calculate the average throughput, the total amount of data transmitted is divided by the total transmission time.

The total data transmitted is determined by subtracting the sequence number of the first TCP segment (1 byte for segment No. 24) from the acknowledged sequence number of the final ACK (32,570 bytes for segment No. 18,895). Thus, the total data transmitted is 32,570−1=32,56932,570 - 1 = 32,56932,570−1=32,569 bytes.

The total transmission time is calculated as the difference between the timestamp of the first TCP segment (0.01814 seconds for segment No. 24) and the timestamp of the last ACK (38.4858 seconds for segment No. 18,895). Therefore, the total transmission time is 38.4858−0.01814=38.46738.4858 - 0.01814 = 38.46738.4858−0.01814=38.467 seconds.

Finally, the average throughput for the TCP connection is computed as 32,56938.467≈846.82\frac{32,569}{38.467} \approx 846.8238.46732,569≈846.82 KB/s.